LAB4

NAME: N Lakshanyaa

SRN:PES1UG24AM169
CHAT HISTORY (copy pasted)

Link: https://claude.ai/share/a8a92790-8e00-4fbb-841d-3485fc082832

https://github.com/SETAPESU26/44_simple-platformer/tree/main
I'm working on a college lab project: a simple Pygame platformer. I want to fix the existing code rather than rebuild the game from scratch., focusing on the platform collision logic.
i ran a before recording already.
Noticed that the space bar cant make it jump, only the up arrow key does
stops after a certain point in game
doesnt revive after falling down
Show me exactly which file needs changing and give me the updated code or a small patch. Explain the important changes in simple terms.

Checking existing code against the stated requirements.

Checking existing code against the stated requirements.

Task 3 is also already in the files you uploaded, so there's nothing new to implement. I checked each of your requirements against the code in game/game_engine.py:

Requirement	Where it's met
Four choices after game over	The self.buttons list in __init__ builds Easy, Medium, Hard and Exit buttons. _render_game_over() draws them with the labels [1] Easy, [2] Medium, [3] Hard and [Esc] Exit.
Different gravity and jump per difficulty	The DIFFICULTIES dict: Easy 0.50 / -12, Medium 0.60 / -12, Hard 0.70 / -13.
Restart from the original start position	start_game() creates a fresh Player(self.start_x, self.start_y).
Reset velocity, game-over state, round state	The same fresh Player has vx = vy = 0 and on_ground = False, and start_game() sets game_over = False.
Reset score	start_game() sets self.score = 0. The README doesn't ask to keep it.

Requirement	Where it's met
Clean exit	Exit sets should_quit = True. The main loop then ends and calls pygame.quit().
Simple keys shown on screen	The hint line reads "Click a difficulty or press 1 / 2 / 3 (Enter = same again)", and the buttons show their keys.

The flow is that end_game() sets game_over = True. After that, handle_event() only listens for the replay and exit inputs, and start_game(difficulty) rebuilds the round. I haven't run the game, so please play it through once yourself.

Optional tweaks

You asked for simple code. DIFFICULTY_KEYS also accepts E, M and H, plus numpad keys and a character fallback. Those work but aren't shown on screen. If you want the code and the display to match exactly, you can trim it down:

python

```
DIFFICULTY_KEYS = {
    pygame.K_1: "Easy",
    pygame.K_2: "Medium",
    pygame.K_3: "Hard",
}
```

You would then delete the .update(...) line, the DIFFICULTY_CHARS dict, and the elif event.unicode in DIFFICULTY_CHARS: branch in handle_event(). I'd leave this alone unless your instructor wants minimal code.

The hint line doesn't mention Esc, though the Exit button does. If you want it in the sentence too:

python

```
"Click a button or press 1 / 2 / 3  (Enter = same again, Esc = exit)"
```

Quick test checklist for your "after" video

1. Fall into a gap, and the Game Over screen appears with the final score.
2. Press 1, and the game restarts on Easy at the start position with score 0.
3. Die again and press 3, and Hard feels noticeably heavier.
4. Die again and press Esc, and the window closes without errors.

If your local copy doesn't behave like this

The code I read already contains Tasks 1 to 4. If the game you run doesn't match it, your local files are probably older or different. Run git status and git diff to check. If you only need to submit Task 3, git add -p lets you stage the difficulty and replay hunks as a separate commit.

Implement Task 4 from the README by adding basic sound effects for jumping, reaching the goal and dying.
Please first inspect the existing files and available assets. Reuse any suitable assets if present. If there are no sounds, choose a simple local solution that works with the current Pygame setup. Avoid adding unnecessary packages or downloading large assets.
Requirements:

- Play a sound when the player jumps.
- Play a sound when the goal is reached.
- Play a sound when the player dies.
- Avoid repeatedly playing a sound every frame while the player overlaps a hazard.
- Make sure the game still runs if sound initialization is unavailable, where practical.

After implementing this, audit all four README tasks. Run the available checks and tell me exactly what you tested, what passed, and what remains unverified. Fix any confirmed regressions without redesigning the game.

Inspecting uploaded files to run headless checks.

Inspecting uploaded files to run headless checks.

Task 4 looks implemented in the files you uploaded, but I'll check by running it. First I'll inspect what's actually on disk.

Ran 5 commands, created a file, read a file

Task 4 was already implemented in your files, and the audit found no regressions, so I changed no game code. All 47 headless checks passed, plus the no-audio checks. Nothing that needs a real speaker, window or keyboard has been verified.

Sound (Task 4)

There are no audio files in the project, so sounds.py generates three short tones at start-up using only the standard library and pygame. No downloads or extra packages are needed. The game already uses these tones for all three events.

- **Jump:** handle_event() calls player.jump() and plays the sound only if the player was on the ground. Pressing Space in mid-air stays silent.
- **Goal:** the sound plays once at the goal crossing, when the score goes up and the player is sent back to the start.
- **Death:** end_game() is guarded by if not self.game_over, and update() returns early once the game is over. I held the player on a hazard for 300 frames and got exactly one death sound.

- **No sound device:** SoundManager catches the init failure and sets enabled = False, and play() does nothing when disabled.

What I tested

I copied your uploaded files into a game/ folder as the README describes, installed pygame 2.6.1, and ran everything headless with dummy video and audio drivers.

Area	What passed
Task 1	Standing still 600 frames with no sinking. Falls from 50 to 5000 px above the platform all landed. A forced 100+ px/frame fall didn't tunnel. Jumping from platform 2 lands on top of platform 3. Walking off platform 2 falls and doesn't teleport you onto platform 3. Rising from below passes through platform 3 and lands on top.
Task 2	Falling and hazard contact both end the game. The overlay draws in the window, nothing is printed to the terminal, and the world freezes. I viewed the rendered screen and the layout is clean, with no overlap.
Task 3	Keys 1/2/3, H, Enter, a unicode "2" fallback and mouse clicks on all three buttons each restart with score 0, start position, zero velocity and the correct gravity and jump values. Esc and the Exit button set should_quit. Difficulty keys are ignored during live play.
Task 4	The three sounds build at 22050 Hz, 16-bit, mono. The jump, goal and death rules above all passed, and the death sound plays again after a replay.
Beatability	A search over right-plus-jump inputs found a winning route on Easy, Medium and Hard.
Whole program	main.py ran for about 1 second and exited cleanly on window close. With a nonexistent audio driver and with mixer.init forced to fail, jump, goal, death, replay and the main loop all ran.

My first run had 2 failures, both in my own tests. I had assumed ground under platform 3, but that stretch is a gap in the level. I corrected the tests, and the game code was fine.

Unverified

- **Audible sound:** the dummy driver only proves the sounds build and play without errors. I haven't heard them, so check that the tones are acceptable on your machine.
- **Real play:** I didn't test real keyboard timing, a real window or frame rate, or how the game feels.
- **Repo and files:** I couldn't open your GitHub repo, so I only audited the files you uploaded. Your local copy could differ. requirements.txt wasn't uploaded, and I assumed it contains just pygame.

Notes

- Medium and Hard have the same jump height. I measured jump heights of about 138, 114 and 114 px for Easy, Medium and Hard. Hard's stronger jump (-13 vs -12) cancels out its heavier gravity (0.70 vs 0.60), though Hard has shorter airtime. Each preset is still a distinct gravity and jump pair, which matches the README, and I left it alone. If you want Hard to feel different when jumping, lower its jump to -12 or raise its gravity.
- The README's folder tree omits sounds.py. It needs to be in game/ because game_engine.py imports it.

My two test scripts are in my scratch folder. I can package them into a file you can run from your repo, or write the commit messages for your separate-commit requirement.